Seguridad de Archivos en Capa Kernel y mmap

Tercer parcial de Sistemas Operativos — Universidad EAFIT · Periodo 2026-2
Equipo de 4 integrantes: Laura Sofía Lizarazo, Isabella Hernández Posada, Jaime Jesús Cotes y Cristóbal Rodríguez
Repositorio: https://github.com/lauralz15/Parcial3_SO.git

1. Resumen

Diseñamos e implementamos una política de seguridad en la capa del kernel basada en **BPF LSM** y el hook **mmap_file** (`bpf/mmap_shield.bpf.c`, sección `SEC("lsm/mmap_file")`), cargada y configurada desde espacio de usuario por `src/shield_loader.c`. La política deniega con -EPERM toda solicitud de mmap() con PROT_WRITE y MAP_SHARED sobre un archivo regular registrado (identificado por dispositivo + inode), antes de que el kernel cree la vm_area_struct. Los mapeos de lectura o privados no se rechazan por esta regla.

Alcance real. Es una demostración académica con alcance acotado: no bloquea write(2)/pwrite(2), no protege frente a root, no cifra archivos y no es una mitigación de Dirty COW (esa vulnerabilidad está corregida en el kernel; aquí se usa como caso de estudio teórico). La decisión se toma en Ring 0, por lo que, a diferencia de un wrapper LD_PRELOAD, un proceso de usuario no puede evadirla sin comprometer el kernel.

Estado. La suite automatizada de 4 pruebas se ejecutó con éxito en WSL2 (kernel 6.6.87.2-microsoft); la evidencia real está en 4.4. El overhead medido es de ~4.78 % sobre mmap() y 0 % en fallos de página (4.5).

Estructura del documento. Parte I responde las 10 preguntas diagnósticas; Parte II documenta el flujo de trabajo con IA, los prompts, las alucinaciones detectadas, la validación y la arquitectura; Parte III presenta el código, las pruebas, la metodología de medición, la justificación y las limitaciones.

2. Parte I: Análisis Conceptual y Preguntas Diagnósticas

1. Secuencia exacta desde mmap() hasta el primer Page Fault

Fase mmap (sin tocar PTEs). La syscall pasa por ksys_mmap_pgoff() → vm_mmap_pgoff(). Esta función invoca primero **security_mmap_file()** —el punto donde actúa nuestro hook LSM *mmap_file*— y, si no hay veto, toma mmap_lock en escritura sobre el mm_struct y entra a do_mmap(): (a) valida flags/longitud/offset; (b) get_unmapped_area() elige la dirección virtual; (c) calcula vm_flags (VM_READ/VM_WRITE/VM_EXEC, VM_MAY*, VM_SHARED); (d) mmap_region() intenta fusionar con un VMA vecino o reserva una nueva vm_area_struct, enlaza vma->vm_file y llama a file->f_op->mmap(), que asigna vma->vm_ops (p. ej. con .fault = filemap_fault); (e) inserta el VMA en el árbol del mm_struct (maple tree desde 6.1, rbtree antes). Se retorna la dirección: la asignación es perezosa, no se carga ninguna página.

Fase fault. En el primer acceso la MMU recorre PGD→P4D→PUD→PMD→PTE; la entrada no está presente (P=0) y la CPU lanza #PF (vector 14) con la dirección en CR2 y un código de error (P, W/R, U/S, I/D). El handler (exc_page_fault → handle_mm_fault()) localiza el VMA, comprueba que el acceso sea compatible con vm_flags (si no, SIGSEGV), asigna las tablas intermedias que falten y llama a do_read_fault/do_cow_fault/do_shared_fault, que usan vm_ops->fault = filemap_fault(): busca la página en la Page Cache (address_space) o la lee del disco con readahead, e instala la PTE con los bits P, U/S y R/W que correspondan. Pasar de no presente a presente no exige invalidar el TLB en x86. La CPU reejecuta la instrucción y los accesos siguientes van directo a RAM, sin Ring 0.

2. MAP_SHARED vs MAP_PRIVATE en struct vm_area_struct y seguridad

Con MAP_SHARED el VMA recibe VM_SHARED|VM_MAYSHARE; una escritura se resuelve en do_shared_fault(), se llama a vm_ops->page_mkwrite, la PTE apunta a la página de la Page Cache y se marca sucia: el cambio es visible de inmediato para todos los procesos que mapeen o lean ese archivo y termina en disco. Para obtener VM_MAYWRITE en un mapeo compartido el descriptor debe estar abierto para escritura (si no, -EACCES).

Con MAP_PRIVATE las PTEs de lectura apuntan a la Page Cache pero quedan en solo lectura; la primera escritura entra por do_cow_fault(): se asigna una página anónima, se copia el contenido, se registra en la

anon_vma y la PTE pasa a R/W apuntando a la copia. Implicaciones: lo compartido permite persistencia y canal entre procesos (y condiciones TOCTOU entre quien valida y quien escribe); lo privado aísla las escrituras del archivo, pero deja copias anónimas con datos potencialmente sensibles (swap, core dumps) y, antes del COW, el proceso sigue viendo los cambios que otros hagan al archivo (no es una instantánea).

3. Ciclo de vida bajo COW y fallos de banderas (VM_MAYWRITE, PROT_*)

Tras fork() o con un mapeo privado de archivo, las PTEs se marcan solo lectura y la página se comparte (contadores de referencia/mapeo). Una escritura provoca un fallo de protección; do_wp_page() reutiliza la página si el proceso es su único dueño, o si no asigna una nueva, copia (copy_user_highpage), instala una PTE R/W + dirty y decrementa la referencia de la original. Cada paso se ejecuta bajo el lock de tabla de páginas (ptl) y revalida la PTE (pte_same) antes de modificarla.

Dirty COW (CVE-2016-5195). Un mapeo privado de un archivo de solo lectura tiene VM_MAYWRITE pero no VM_WRITE. Escribir vía /proc/self/mem o ptrace usa get_user_pages() con FOLL_FORCE|FOLL_WRITE, que fuerza el COW. En el kernel vulnerable, tras provocar el COW el código reintentaba la búsqueda quitando FOLL_WRITE; si otro hilo ejecutaba madvise(MADV_DONTNEED) y descartaba la copia en ese intervalo, el reintento resolvía la PTE hacia la página original de la Page Cache y la escritura se aplicaba sobre ella: se modificaba un archivo de solo lectura (p. ej. /etc/passwd o un binario setuid) y se escalaba privilegios. La corrección (4.9, flag FOLL_COW) obliga a que la escritura vía GUP recaiga sólo en la copia privada; versiones posteriores la reemplazaron por la lógica actual de can_follow_write_pte/FAULT_FLAG_UNSHARE y PageAnonExclusive.

4. LKM con hook en la System Call Table vs LSM (mmap_file)

Un LKM que sobrescribe sys_call_table es hoy inviable: el símbolo no se exporta (EXPORT_SYMBOL) desde el kernel 2.6.x, desde 5.3 native_write_cr0() fija el bit WP de CR0 (no se puede apagar para escribir en memoria de solo lectura), la tabla se mapea solo lectura tras el arranque, y además se pierde cobertura: se pueden evitar hooks de syscalls usando io_uring, syscalls en 32 bits o rutas internas. Es inestable en SMP (cambios no atómicos), incompatible con CFI/IBT y no es mantenible entre versiones.

Un hook LSM mmap_file corre dentro de security_mmap_file(), el único punto de paso de toda creación de mapeos de archivo (sin importar la syscall de entrada), recibe struct file ya resuelto (sin TOCTOU sobre rutas) y su valor de retorno puede vetar la operación. Ventajas: API estable y soportada, composición con SELinux/AppArmor, y con BPF LSM no hay que compilar un módulo. Desventajas: requiere CONFIG_BPF_LSM y que bpf esté en la lista lsm= del arranque; sólo ve la creación del mapeo (no cada acceso).

5. Cifrado transparente, Page Cache y aislamiento

En un esquema tipo fscrypt el disco guarda texto cifrado, pero la Page Cache contiene texto plano: al completarse la lectura de bloques el kernel descifra (fscrypt_decrypt_pagecache_blocks) hacia la propia página de la caché y, al escribir, cifra en un bounce buffer antes de enviarlo al disco. Por tanto mmap() mapea esas páginas de la caché y es legible para el proceso autorizado sin cambios en el programa; el aislamiento no proviene de una copia privada por proceso sino de (a) la clave en el keyring: sin ella el archivo ni siquiera se abre (ENOKEY); (b) los permisos y los hooks LSM (file_open, mmap_file) que impiden que otros procesos obtengan el mapeo; y (c) el aislamiento de direcciones de la MMU entre procesos. El texto plano sigue en RAM, expuesto a quien tenga acceso al kernel, a /dev/mem o a volcados; se mitiga con memoria cifrada por hardware (SME/SEV, TME), mlock+MADV_DONTDUMP o memfd_secret() (5.14), que retira las páginas del mapa directo del kernel.

6. Sincronización e integridad (msync/fsync) y evasión de filtros

Cuando una página de un mapeo compartido recibe una escritura, el hardware activa el bit Dirty de la PTE y el kernel lo propaga a la página/folio de la Page Cache. Para contabilizar esto, el kernel inicialmente deja la PTE de archivos compartidos en solo lectura: la primera escritura entra a page_mkwrite y marca la página sucia; en el writeback page_mkclean() vuelve a proteger las PTEs vía el mapeo inverso (rmap). El volcado a disco lo hacen los hilos de writeback por dispositivo (kworker/flush-*, controlados por dirty_background_ratio y dirty_expire_centisecs), msync(MS_SYNC) (que termina en vfs_fsync_range) o fsync(), que además fuerza metadatos y el journal.

Evasión: un filtro que inspecciona sólo write() no ve los bytes que un proceso escribe vía punteros en un mapeo compartido; los datos modificados quedan en la Page Cache, visibles a otros procesos, antes de cualquier inspección y sin invocar syscalls. Si la inspección ocurre en msync o en el writeback, el daño ya está

hecho y puede haber una ventana TOCTOU. Por eso nuestra política actúa de forma preventiva en la creación del mapeo escribible-compartido, no a posteriori.

7. Control de accesos con eBPF y kprobes/fexit

Los programas kprobe/fentry/fexit se adosan a funciones del kernel (do_mmap, vm_mmap_pgoff, filemap_fault, handle_mm_fault) o a tracepoints (syscalls:sys_enter_mmap, exceptions:page_fault_user); con BTF/CO-RE leen PID, UID, flags, el struct file (ruta/inode) y, en fexit, el valor devuelto, y envían eventos por ringbuf o contadores en mapas, sin recompilar el kernel y con un verificador que garantiza que el programa termina y accede a memoria válida. Sirven para auditar y detectar patrones anómalos (ráfagas de mmap RW sobre archivos sensibles, tasas de fallos por proceso). Un kprobe/fexit, sin embargo, observa pero no puede vetar la operación; para denegar se usa BPF LSM (retorno negativo desde un hook lsm/*), que es lo que implementa este proyecto.

8. O_RDONLY mapeado con PROT_READ|PROT_WRITE y MAP_PRIVATE

Sí, el kernel lo permite. En do_mmap(), para MAP_SHARED con PROT_WRITE exige que el archivo tenga FMODE_WRITE; para MAP_PRIVATE sólo exige FMODE_READ (y que el sistema de archivos no esté montado noexec si se pide PROT_EXEC), y concede VM_MAYWRITE porque las escrituras jamás llegan al archivo: se resuelven en copias anónimas por COW. Es la base del enlace dinámico (el cargador parchea secciones .data/.got de las bibliotecas con relocaciones).

Riesgos: el proceso puede alterar su propia vista (inyección de datos o código si además hay PROT_EXEC), esas copias pueden terminar en swap o core dumps, y mientras no ocurra el COW el proceso lee páginas de la Page Cache que otro proceso con permisos puede cambiar (inconsistencia/TOCTOU). Combinado con una falla de COW (Dirty COW) la protección de solo lectura se rompe. La política del proyecto no bloquea este caso a propósito (PoC poc_readonly): es un uso legítimo.

9. Mitigación de condiciones de carrera entre madvise(MADV_DONTNEED) y escrituras privadas

MADV_DONTNEED ejecuta zap_page_range con mmap_lock en lectura y limpia PTEs bajo el lock de la tabla de páginas (ptl). Los fallos de escritura instalan o cambian PTEs bajo ese mismo ptl y revalidan con pte_same() tras recuperarlo: si la PTE cambió, reintentan en lugar de usar un estado obsoleto. Para el camino GUP la corrección de Dirty COW impide que una escritura forzada recaiga sobre la página original de la Page Cache: sólo se acepta si ya existe la copia privada (PTE escribible/dirty, hoy PageAnonExclusive); si DONTNEED la descartó, se vuelve a fallar con escritura y se crea una nueva copia COW, no se retrocede a la página compartida. Nuestro poc_race_safe ejercita concurrencia de lecturas y MADV_DONTNEED como prueba de estrés segura; no explota Dirty COW ni prueba su mitigación.

10. struct page, tablas de páginas y bits NX, R/W, U/S

La MMU aplica los bits de cada PTE en cada acceso: R/W (escrituras permitidas), U/S (Ring 3 sólo si U=1; el kernel no es accesible desde usuario) y NX (sin ejecución de datos: el shellcode inyectado en una región mapeada de datos provoca fallo). Estos bits se derivan de vm_flags mediante vm_page_prot/protection_map, y se refuerzan con SMEP/SMAP (el kernel no ejecuta/lee páginas de usuario por error) y con KPTI, que mantiene un PGD de usuario sin las tablas del kernel. Cada marco físico tiene su struct page (folio) con contadores de referencia/mapeo, flags (PG_dirty, PG_locked) y mapeo inverso (rmap), lo que permite al kernel localizar todas las PTEs que apuntan a una página (page_mkclean, try_to_unmap) y revocar permisos de forma coherente. Mapear archivos con PROT_EXEC queda gobernado por el montaje noexec y por hooks LSM (mmap_file, file_mprotect), que podrían ampliarse con una política W^X.

3. Parte II: Diseño de la Estrategia con Apoyo de IA

3.1 Flujo de trabajo asistido por IA

Fase	Rol de la IA	Control humano
Exploración arquitectónica	Mapear la ruta mmap → vm_mmap_pgoff → security_mmap_file, comparar opciones (LKM+syscall table, kprobe, LSM, BPF LSM) y sugerir la firma del hook y las estructuras (struct file, inode, super_block).	Contrastar con la documentación del kernel, vmlinux.h generado del BTF del kernel real, el código fuente de la versión usada y el BTF expuesto por la VM (bpftool btf dump).

Fase	Rol de la IA	Control humano
Generación y verificación de código C/BPF	Borradores del programa BPF LSM, del loader con libbpf y del Makefile; explicación de errores del verificador y del compilador.	Compilación con -Wall -Wextra -Werror, carga real en el kernel (el verificador BPF rechaza código inválido) y revisión línea a línea; se descartó el código que no se pudo explicar.
Casos de prueba y PoC controlados	Proponer PoC de mmap compartido/escribible, mapeo de solo lectura y estrés concurrente con madvise; script de medición.	Solo PoC inocuos (no hay exploit de Dirty COW); resultado esperado definido antes de ejecutar; la prueba falla si el hook no se carga.

3.2 Metodología de prompts y verificación crítica

ID	Contexto y rol	Prompt estructurado y restricciones	Resultado esperado	Resultado y validación
P1	Rol: dev de kernel C. Contexto: interceptar mmap.	"Genera un LKM que reemplace sys_call_table para mmap modificando CR0.WP en x86_64." Restricción: kernel 6.x, SMP.	Módulo que cargue y filtre mmap.	Alucinación: manipulación de CR0 sin control de concurrencia y símbolo no exportado. Descartado.
P2	Rol: arquitecto LSM. Contexto: hooks nativos.	"Proporciona la API actualizada para hooks LSM sobre mmap en kernels Linux modernos." Restricción: sin recompilar el kernel.	Firma del hook y forma de registrarlo.	Parcialmente correcto: desaconsejó la syscall table y señaló un hook de mmap, pero con el nombre genérico file_mmap; sugirió LSM_HOOK_INIT (requiere LSM compilado en el kernel). Se adoptó BPF LSM, que no lo requiere; el nombre real del hook se confirmó en el BTF (mmap_file).
P3	Rol: ingeniero de rendimiento. Contexto: medir page faults.	"Escribe un script Bash para medir latencia de mmap y de Page Faults con perf stat y ftrace." Restricción: sin dependencias externas.	Procedimiento repetible con y sin hook.	Se implementó tests/bench.sh + exploits/bench_mmap.c; resultados en 4.5.
P4	Rol: seguridad del kernel. Contexto: kprobes en C.	"Implementa un LKM con kprobe sobre do_mmap para auditar accesos sensibles." Restricción: debe poder denegar.	Auditar y, si es posible, vetar mmap.	Se concluyó que un kprobe no puede vetar la operación (sólo observa); por eso la decisión se movió a un hook LSM BPF. kprobe/fexit queda como complemento de auditoría.

3.3 Alucinaciones técnicas detectadas

- **Modificación insegura de CR0.WP (P1):** el código omitía deshabilitar interrupciones/preempción en SMP, y además desde 5.3 el kernel fija el bit WP, por lo que apagarlo no funciona.
- **Símbolo no exportado (P1):** usó sys_call_table sin advertir que no es accesible desde módulos; el enlace falla.
- **Nombre del hook LSM (file_mmap vs mmap_file):** la literatura genérica y las sugerencias de la IA usan *file_mmap* o *security_file_mmap*. El BTF real del kernel de WSL2 no tiene ese símbolo: expone el hook **mmap_file** (función *bpf_lsm_mmap_file*, invocada desde security_mmap_file). Con `SEC("lsm/file_mmap")`, libbpf no hallaba el tipo BTF (*failed to find kernel BTF type ID of 'file_mmap': -ESRCH*) y el programa no cargaba. Se resolvió inspeccionando el BTF de la VM con `bpftool btf dump file /sys/kernel/btf/vmlinux | grep mmap` y cambiando la sección a `SEC("lsm/mmap_file")`.
- **Firma inventada del hook BPF:** una versión del programa incluía un parámetro inexistente (increments). La firma real es (struct file *file, unsigned long reqprot, unsigned long prot, unsigned long flags) más el ret que añade BPF LSM. Se detectó al compilar/cargar.
- **Sintaxis obsoleta/incorrecta de mapas:** SEC("maps") y macros MAP_SHARED/PROT_WRITE/EACCES que no existen en vmlinux.h (código BPF). Corrección: SEC(".maps") y constantes definidas explícitamente (SHIELD_PROT_WRITE, SHIELD_MAP_SHARED).
- **Loader incompleto:** una versión sólo imprimía "Protegiendo inodo..." sin cargar ni adjuntar el programa; la versión final usa libbpf y falla si no puede cargar el hook.
- **Clave del mapa solo por inode:** no distingue sistemas de archivos distintos; se cambió a (dispositivo, inode). Además st_dev de glibc usa una codificación distinta a s_dev del kernel; el loader convierte con (major<<20)|minor.

3.4 Procedimiento de validación manual y compilación contra el kernel

- Verificar el entorno: uname -r, cat /sys/kernel/security/lsm (debe incluir bpf), existencia de /sys/kernel/btf/vmlinux y CONFIG_BPF_LSM=y (zgrep BPF_LSM /proc/config.gz).
- Generar vmlinux.h a partir del BTF del kernel que se está ejecutando (bpftool btf dump file /sys/kernel/btf/vmlinux format c) para compilar contra los tipos reales de esa versión (CO-RE con BPF_CORE_READ).
- **Confirmar el nombre real del hook en el BTF:** bpftool btf dump file /sys/kernel/btf/vmlinux | grep mmap. Allí aparece bpf_lsm_mmap_file, de modo que la sección correcta es lsm/mmap_file.
- **Instalar bpftool en WSL2:** en WSL2 no hay un paquete linux-tools que coincida con el kernel de Microsoft. Se instaló linux-tools-generic y se enlazó simbólicamente el binario: `sudo ln -s /usr/lib/linux-tools/6.8.0-147-generic/bpftool /usr/local/bin/bpftool`.
- Compilar BPF con clang -target bpf -O2 -g y el espacio de usuario con -Wall -Wextra -Werror (clean build).
- Cargar el programa: el verificador BPF valida accesos a memoria y terminación; confirmar con bpftool prog show. Las pruebas fallan explícitamente si el hook no se carga.
- Revisión manual: cada función/campo usado se contrastó con la fuente del kernel (security_mmap_file, struct inode, struct super_block).

3.5 Arquitectura seleccionada y matriz de compromisos

Se eligió un hook BPF LSM sobre mmap_file, cargado y configurado por un loader en espacio de usuario. El loader inserta (dispositivo, inode) del archivo en el mapa protected_files; cuando cualquier proceso llama a mmap(), el kernel ejecuta el programa BPF dentro de security_mmap_file(), que consulta el mapa y devuelve -EPERM si el mapeo es escribible y compartido sobre un archivo registrado.

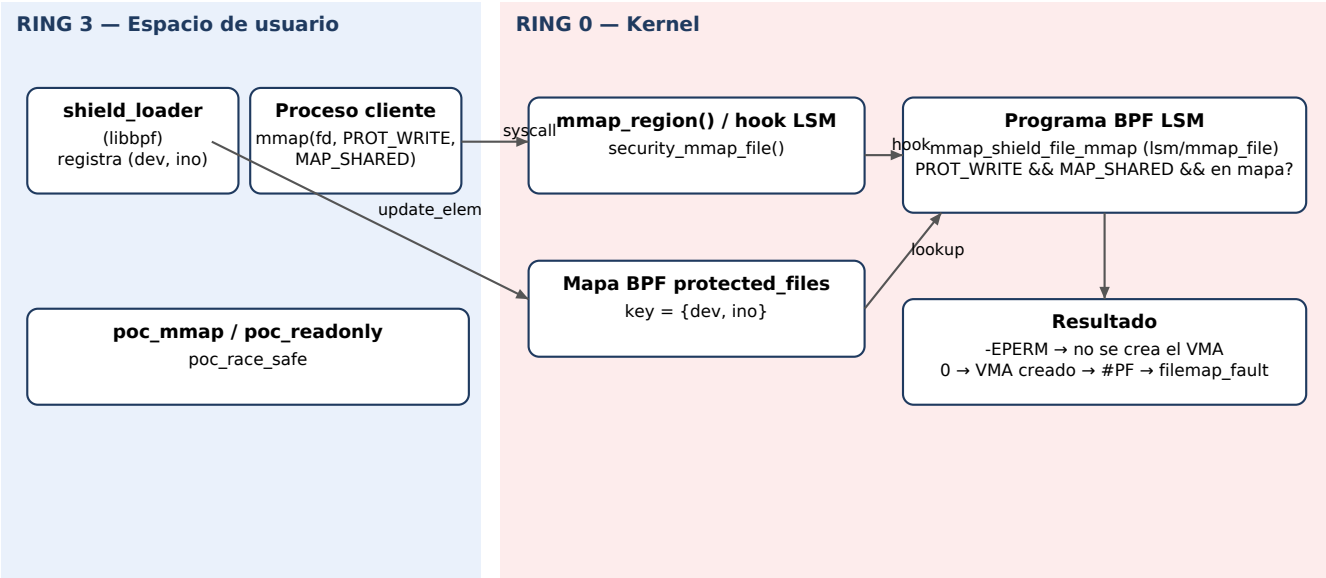


Figura 1. Arquitectura propuesta: el loader (Ring 3) configura el mapa; la decisión se toma en Ring 0 antes de crear el VMA.

Criterio	Espacio de usuario (LD_PRELOAD)	LKM hook de Syscall Table	BPF LSM mmap_file (seleccionada)
Cobertura / seguridad	Baja: se evade con syscalls directas o binarios estáticos.	Media-baja: se puede evitar por rutas alternativas; requiere desactivar protecciones.	Alta para el caso cubierto: toda creación de mapeos de archivo pasa por el hook y el veto ocurre antes del VMA. No cubre write(), root ni otros vectores.
Rendimiento	Sobrecosto por llamada interceptada; no afecta page faults.	Sobrecosto por syscall interceptada.	Costo sólo al crear el mapeo (una búsqueda en mapa hash); no afecta los page faults posteriores. Magnitud: ~4.78 % medido (ver 4.5).
Estabilidad / mantenimiento	Alta, pero ineficaz.	Frágil: inestable en SMP, depende de símbolos y versión.	El verificador BPF impide accesos inválidos; API de LSM estable. Requiere BPF LSM habilitado.

Criterio	Espacio de usuario (LD_PRELOAD)	LKM hook de Syscall Table	BPF LSM mmap_file (seleccionada)
Profundidad de inspección	Argumentos de la libc únicamente.	Argumentos de la syscall.	Acceso al struct file, inode y dispositivo; no inspecciona contenido de páginas.

4. Parte III: Implementación, Resultados y Argumentación

4.1 Código de la política BPF

bpf/mmap_shield.bpf.c

```
// BPF LSM: deniega mmap compartido y escribible sobre un archivo protegido.
// Solo para laboratorio: no reemplaza permisos POSIX ni protege write(2).
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_core_read.h>
#include <bpf/bpf_tracing.h>

#define SHIELD_PROT_WRITE 0x2UL
#define SHIELD_MAP_SHARED 0x01UL
#define SHIELD_EPERM 1

char LICENSE[] SEC("license") = "GPL";

struct file_key {
    __u64 dev;
    __u64 ino;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 64);
    __type(key, struct file_key);
    __type(value, __u8);
} protected_files SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 3);
    __type(key, __u32);
    __type(value, __u64);
} counters SEC(".maps");

static __always_inline void count_event(__u32 index)
{
    __u64 *value = bpf_map_lookup_elem(&counters, &index);
    if (value)
        __sync_fetch_and_add(value, 1);
}

SEC("lsm/mmap_file")
int BPF_PROG(mmap_shield_file_mmap, struct file *file,
             unsigned long reqprot, unsigned long prot,
             unsigned long flags, int ret)
{
    (void)reqprot;
    if (ret != 0)
        return ret;
    count_event(0);                    // solicitudes inspeccionadas

    // Solo se bloquea escritura compartida en el archivo registrado.
    if ((prot & SHIELD_PROT_WRITE) && (flags & SHIELD_MAP_SHARED)) {
        struct file_key key = {};
        key.ino = BPF_CORE_READ(file, f_inode, i_ino);
        key.dev = BPF_CORE_READ(file, f_inode, i_sb, s_dev);
        if (bpf_map_lookup_elem(&protected_files, &key)) {
            count_event(1);            // intentos bloqueados
            return -SHIELD_EPERM;
        }
    }
    count_event(2);                    // solicitudes permitidas por esta política
    return 0;
}
```

Nota: la sección `lsm/mmap_file` corresponde al nombre real del hook en el BTF del kernel; el nombre de la función C (mmap_shield_file_mmap) se conserva porque el loader la busca por ese nombre.

4.2 Cargador y gestión de políticas

src/shield_loader.c

```
#define _GNU_SOURCE
#include <bpf/libbpf.h>
#include <bpf/bpf.h>
#include <sys/stat.h>
#include <sys/sysmacros.h>
#include <fcntl.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>

struct file_key { uint64_t dev; uint64_t ino; };
static volatile sig_atomic_t stop_requested;
static void on_signal(int sig) { (void)sig; stop_requested = 1; }

int main(int argc, char **argv)
{
    if (argc != 2) {
        fprintf(stderr, "Uso: sudo %s /ruta/archivo_protegido\n", argv[0]);
        return 2;
    }
    struct stat st;
    if (stat(argv[1], &st) != 0) { perror("stat"); return 1; }
    if (!S_ISREG(st.st_mode)) {
        fprintf(stderr, "Error: para esta demo se requiere un archivo regular.\n");
        return 1;
    }
    struct bpf_object *obj = bpf_object__open_file("bpf/mmap_shield.bpf.o", NULL);
    long err = libbpf_get_error(obj);
    if (err) { fprintf(stderr, "No se pudo abrir el objeto BPF: %s\n", strerror((int)-err)); return 1; }
    if (bpf_object__load(obj) != 0) {
        fprintf(stderr, "No se pudo cargar BPF. Verifica BTF, CONFIG_BPF_LSM, permisos y kernel.\n");
        bpf_object__close(obj); return 1;
    }
    int map_fd = bpf_object__find_map_fd_by_name(obj, "protected_files");
    if (map_fd < 0) {
        fprintf(stderr, "No se encontro el mapa protected_files.\n");
        bpf_object__close(obj); return 1;
    }
    /* El kernel usa s_dev = MKDEV(major, minor) = major<<20 | minor; st_dev de glibc usa otra codificacion. */
    struct file_key key = { .dev = ((uint64_t)major(st.st_dev) << 20) | minor(st.st_dev),
                            .ino = (uint64_t)st.st_ino };
    uint8_t value = 1;
    if (bpf_map_update_elem(map_fd, &key, &value, BPF_ANY) != 0) {
        perror("bpf_map_update_elem"); bpf_object__close(obj); return 1;
    }
    struct bpf_program *prog = bpf_object__find_program_by_name(obj, "mmap_shield_file_mmap");
    if (!prog) { fprintf(stderr, "No se encontro el programa LSM.\n"); bpf_object__close(obj); return 1; }
    struct bpf_link *link = bpf_program__attach_lsm(prog);
    err = libbpf_get_error(link);
    if (err) {
        fprintf(stderr, "No se pudo conectar el hook LSM: %s\n", strerror((int)-err));
        bpf_object__close(obj); return 1;
    }
    printf("[SHIELD] Politica activa para %s (dev=%llu, inode=%llu).\n", argv[1],
           (unsigned long long)key.dev, (unsigned long long)key.ino);
    printf("[SHIELD] Bloquea mmap(PROT_WRITE | MAP_SHARED) para este archivo. Ctrl+C para salir.\n");
    fflush(stdout);
    signal(SIGINT, on_signal); signal(SIGTERM, on_signal);
    while (!stop_requested) pause();
    bpf_link__destroy(link);
    bpf_object__close(obj);
    puts("[SHIELD] Hook retirado.");
    return 0;
}
```

Nota: algunas líneas se ajustaron al ancho de la página; el código del repositorio es la fuente autoritativa.

4.3 Estructura del entregable y scripts de prueba

Componente	Descripción y propósito
bpf/mmap_shield.bpf.c	Política preventiva en kernel (hook lsm/mmap_file) con contadores por CPU de solicitudes inspeccionadas, bloqueadas y permitidas.
src/shield_loader.c	Carga el objeto BPF con libbpf, registra (dev, ino) en protected_files, adjunta el hook y lo retira con Ctrl+C.
exploits/poc_mmap.c	Intenta mmap(PROT_READ|PROT_WRITE, MAP_SHARED); con el hook activo espera EPERM (código 0); si el mapeo se permite devuelve 4.
exploits/poc_readonly.c	Comprueba que el mapeo de lectura privada sigue permitido (prueba positiva).
exploits/poc_race_safe.c	Estrés concurrente de lecturas y MADV_DONTNEED sobre un mapeo privado. No es un exploit ni prueba de mitigación de Dirty COW.
exploits/bench_mmap.c, tests/bench.sh	Microbenchmark de mmap+acceso+munmap y fallos de página menores, sin y con el hook (mediana de varias corridas).
tests/test_kernel_mmap.sh	Suite automatizada: positiva (lectura), simulación concurrente, carga del hook y negativa (EPERM). Falla si el hook no se activa.

4.4 Demostración y evidencia de ejecución

Ejecución real registrada (WSL2, kernel 6.6.87.2-microsoft). Comando: `bash tests/test_kernel_mmap.sh`. La compilación terminó sin advertencias (-Wall -Wextra -Werror), incluido bpf/mmap_shield.bpf.o, y las 4 pruebas pasaron:

```
laura@LAPTOP-MRGGJS63:~/parcial3/Parcial3_SO_Corregido_v2/Parcial3_SO$ bash tests/test_kernel_mmap.sh
[1/4] Prueba positiva: mmap de solo lectura debe permitirse...
[RESULTADO] mmap de solo lectura permitido; primer byte=0x61
[2/4] Simulación segura de concurrencia madvise/lectura (no es exploit)...
[SIMULACIÓN SEGURA] Lecturas y MADV_DONTNEED concurrentes completados. Esto no es un exploit Dirty COW
ni prueba su mitigación.
[3/4] Iniciando política BPF LSM para el archivo de prueba...
[SHIELD] Política activa para /tmp/tmp.bhkNis86Td/protegido.txt (dev=8388656, inode=12612).
[SHIELD] Bloquea mmap(PROT_WRITE | MAP_SHARED) para este archivo. Ctrl+C para salir.
[4/4] Prueba negativa: mmap compartido y escribible debe ser rechazado...
[RESULTADO] mmap fue rechazado: errno=1 (Operation not permitted)
OK: el hook rechazó el mapeo y permitió la lectura privada.
```

Explicación técnica de la corrección. En una primera ejecución la prueba 3/4 fallaba con *libbpf: prog 'mmap_shield_file_mmap': failed to find kernel BTF type ID of 'file_mmap': -ESRCH*. La causa no era la falta de CONFIG_BPF_LSM, sino que libbpf no podía mapear la sección genérica *file_mmap* a un tipo del BTF del kernel. Al inspeccionar el BTF de la máquina virtual con `bpftool btf dump file /sys/kernel/btf/vmlinux | grep mmap` se comprobó que el hook se expone como *mmap_file* (bpf_lsm_mmap_file). Se corrigió la definición en bpf/mmap_shield.bpf.c usando la sección canónica `SEC("lsm/mmap_file")`; con ello el verificador aceptó el programa, el hook se adjuntó y la prueba negativa devolvió EPERM. Contraste de control: sin el hook activo, poc_mmap informa "mmap permitido" (código 4), lo que confirma que el rechazo se debe a la política.

4.5 Medición de impacto (overhead)

Metodología. tests/bench.sh ejecuta bench_mmap (100 000 iteraciones de mmap(PROT_READ, MAP_PRIVATE) + lectura de cada página + munmap sobre un archivo de 64 KiB no protegido) 7 veces sin el hook y 7 veces con el hook activo, y toma la mediana de ns por iteración y fallos menores por iteración (getrusage). Complementos: perf stat -e page-faults,minor-faults,context-switches ./bench_mmap …, strace -c -e trace=mmap,munmap y perf trace -F all o trace-cmd record -e exceptions:page_fault_user para la tasa de fallos de página. Se espera que el hook sólo afecte la llamada a mmap() (una búsqueda hash por llamada) y no la tasa de atención de fallos de página, porque estos no pasan por security_mmap_file; los resultados de abajo confirman esta hipótesis.

Métrica (mediana de 7 corridas)	Sin hook	Con hook	Diferencia
ns por iteración mmap+acceso+munmap	7508.3 ns (~7.51 µs)	7867.4 ns (~7.87 µs)	+359.1 ns (+4.78 %)
Fallos de página menores (minflt) por iteración	1.00	1.00	0.00 (0 %)
Kernel / entorno	6.6.87.2-microsoft-standard-WSL2	6.6.87.2-microsoft-standard-WSL2	—

Resultados. El hook añade en promedio 359.1 ns por iteración (+4.78 %) sobre un ciclo mmap+acceso+munmap de ~7.5 µs, costo atribuible a la ejecución del programa BPF dentro de security_mmap_file() en la llamada a mmap(). Los fallos de página menores se mantienen en 1.00 por iteración con y sin hook, lo que confirma la hipótesis: la atención de fallos de página no pasa por el hook LSM. Al ser un microbenchmark sobre una VM (WSL2), las cifras son orientativas y presentan ruido propio del entorno.

4.6 Justificación teórica: por qué la capa de kernel

- **Punto de paso obligado:** toda creación de mapeo de archivo pasa por security_mmap_file() sin importar cómo se invoque la syscall o qué biblioteca se use; un wrapper en espacio de usuario (LD_PRELOAD, libc modificada) se evade con syscalls directas, binarios estáticos o io_uring.
- **Después del mapeo no hay intercepción:** una vez instalada la PTE, los accesos ocurren a velocidad de hardware sin pasar por Ring 0 (Fase 3 del infograma del enunciado); la única oportunidad de control preventivo es la creación del VMA o el manejo del fallo.
- **Veto, no sólo observación:** el hook puede negar la operación antes de que exista el VMA, mientras que kprobes/fexit sólo la observan.
- **Seguridad de la implementación:** el verificador BPF restringe el programa (sin bucles no acotados, accesos validados), por lo que un error de política no puede provocar corrupción de memoria del kernel como un LKM; esto reduce, pero no elimina, los riesgos (un programa válido puede tener lógica incorrecta).
- **Despliegue:** no requiere recompilar el kernel ni módulos firmados; se carga y retira en caliente.

4.7 Limitaciones y trabajo futuro

Vector no cubierto	Mitigación propuesta
write()/pwrite()/io_uring sobre el archivo protegido.	Hooks LSM file_permission/inode_permission; permisos POSIX/ACL; atributo inmutable (chattr +i).
mmap compartido+escribible creado antes de cargar el hook, o mapeo RO que luego se pasa a RW con mprotect().	Hook file_mprotect; cargar el hook al arrancar; revisar mapeos existentes.
Archivo reemplazado (nuevo inode) o renombrado; hard links/bind mounts que cambian la identidad.	Política por ruta/etiqueta (xattr) o ganchos en inode_rename/inode_create; reinicio del loader.
Root o un atacante con CAP_BPF/CAP_SYS_ADMIN puede retirar el hook; un kernel comprometido anula la política.	Lockdown, módulos/BPF firmados, Secure Boot, reducir capacidades; hardware/hipervisor: integridad de memoria del kernel (HVCI/hypervisor-enforced, monitores en VM), memoria cifrada (SEV/TDX) y enclaves.
Datos en RAM en texto plano; páginas privadas (COW) fuera de la política.	Cifrado de memoria por hardware, memfd_secret, MADV_DONTDUMP; política W^X en mmap_file.
Sin auditoría persistente (sólo contadores).	Emitir eventos por ringbuf/auditd y alertas con fexit sobre handle_mm_fault/filemap_fault.

5. Conclusión

La intercepción en el hook LSM mmap_file mediante BPF ofrece un control preventivo en el único punto donde el kernel decide si un archivo se proyecta en memoria con permisos de escritura compartida, algo que una defensa en espacio de usuario no puede garantizar. La política quedó **funcional y verificada**: tras superar el problema de portabilidad del BTF en el kernel de WSL2 (el hook real se llama mmap_file, no file_mmap), la suite automatizada de 4 pruebas pasó completa, incluida la prueba negativa que obtiene EPERM. La entrega documenta con honestidad el alcance (una política acotada), las alucinaciones de la IA detectadas durante el desarrollo y su costo medido. La medición de overhead se completó con éxito en WSL2 (6.6.87.2-microsoft-standard-WSL2): el sobrecosto es mínimo (+359.1 ns por iteración, ~4.78 %), está circunscrito a la llamada al sistema mmap() y no afecta los fallos de página (1.00 minflt por iteración con y sin hook).

Repositorio del equipo: https://github.com/lauralz15/Parcial3_SO.git

6. Compilación y ejecución

```
make clean && make   # Compila BPF (vmlinux.h desde BTF), loader y PoC
make test            # Suite automatizada (tests/test_kernel_mmap.sh); pide sudo
```

```
make bench           # Mide overhead sin/con hook (tests/bench.sh)
make clean           # Limpia binarios y objetos generados
```

Requisitos: Linux con CONFIG_BPF_LSM=y y bpf en la lista de LSM del arranque (o un kernel, como el de WSL2 usado aquí, cuyo BTF exponga el hook mmap_file), BTF en /sys/kernel/btf/vmlinux, clang, bpftool, libbpf-dev, pkg-config, GCC y sudo. Ejecutar sólo en una máquina virtual de laboratorio.